

司机端测试计划

1. 概述

本文档为校车管理系统司机端模块的测试计划，涵盖 Widget 测试和单元测试，目标为验证司机端核心功能的正确性和稳定性。

2. 测试范围

模块	优先级	说明
司机端登录与工作台展示	P0	登录流程、工作台布局、关键信息展示
统计看板	P0	日/周/月维度切换、准点率展示
任务列表展示	P0	任务卡片信息、乘客名单展开
开始任务与位置上报	P0	任务启动、位置上报机制
完成任务	P0	任务完成流程、状态变更
边界情况	P1	异常操作、状态冲突处理
DriverTaskState 单元测试	P0	State getter 和 copyWith 方法
DriverTaskController 单元测试	P0	Controller 公开方法
模型与枚举测试	P1	Model 和 Enum 字段验证

3. 测试环境

项目	内容
测试框架	Flutter Test (flutter_test)
状态管理	Riverpod (ProviderContainer 隔离测试)
模拟数据	硬编码 Mock 数据
测试文件	test/driver_test.dart

4. 测试策略

Widget 测试策略：

- 使用 pumpWidget 构建组件树
- 使用 pumpAndSettle 等待动画完成
- 使用 find.byKey 定位特定组件
- 使用 ensureVisible 确保组件在视口中
- 使用 scrollUntilVisible 处理滚动场景

单元测试策略：

- 使用 ProviderContainer 隔离测试 Riverpod 状态
- 直接调用 Controller 方法验证返回值

- 验证 State 变更后的字段值
- 使用 addTearDown 清理测试资源

5. 测试用例清单

5.1 司机端登录与工作台展示（7 个）

编号	用例名称	验证点
TC-01	登录成功后显示工作台	工作台标题、统计看板、实时位置共享、今日派车任务
TC-02	工作台显示司机姓名	AppBar 中显示"王师傅"
TC-03	工作台显示待发任务数量	"待发任务"标签
TC-04	工作台显示今日乘客数量	"今日乘客"标签
TC-05	初始位置状态为未开启	"未开启"、"位置状态"
TC-06	显示任务列表中的路线名称	三条路线名称
TC-07	显示暂无位置上报	"暂无位置上报"提示

5.2 统计看板（4 个）

编号	用例名称	验证点
TC-08	默认显示日维度数据	完成车次 4 趟、乘客 38 人、里程 42.6 km
TC-09	可切换到周维度	24 趟、236 人、286.4 km
TC-10	可切换到月维度	98 趟、1034 人、1198.5 km
TC-11	显示准点率进度条	"准点率 98%"、LinearProgressIndicator

5.3 任务列表展示（8 个）

编号	用例名称	验证点
TC-12	显示路线名称和车次号	路线名 · 车次号格式
TC-13	显示起止站点	起点 → 终点格式
TC-14	显示车牌号	"校 A · 1028"
TC-15	显示乘客数量	"X 名乘客"
TC-16	显示行驶里程	"X.X km"
TC-17	显示待发任务状态标签	"待发任务"
TC-18	可展开查看乘客名单	乘客姓名和尾号
TC-19	乘客名单显示上车点和座位号	上车点名称

5.4 开始任务与位置上报（7 个）

编号	用例名称	验证点
TC-20	点击开始任务按钮可启动任务	状态变为"行程中"、"共享中"
TC-21	开始任务后自动上报一次位置	上报次数 1、经纬度显示
TC-22	模拟上报按钮可手动触发位置上报	上报次数增加
TC-23	多次模拟上报位置坐标会变化	经纬度变化
TC-24	开始任务后显示车辆 ID	"bus-001"
TC-25	开始任务后显示速度信息	"km/h"

5.5 完成任务（3 个）

编号	用例名称	验证点
TC-26	到站完成按钮可结束当前任务	状态变为"已完成"、"未开启"
TC-27	完成任务后位置上报停止	上报按钮被禁用
TC-28	完成任务后显示提示消息	SnackBar 提示"任务已完成"

5.6 边界情况 (5 个)

编号	用例名称	验证点
TC-29	未开始任务时模拟上报按钮不可用	onPressed 为 null
TC-30	未开始任务时到站完成按钮不可用	onPressed 为 null
TC-31	不能同时开始两个任务	第二个任务按钮被禁用
TC-32	已完成任务显示为不可操作状态	"任务已完成"按钮
TC-33	完成任务后可以开始另一个任务	新任务可启动

5.7 DriverTaskState 单元测试 (9 个)

编号	用例名称	验证点
TC-34	pendingTaskCount 返回待发车任务数量	返回 1
TC-35	todayPassengerCount 返回所有乘客总数	返回 3
TC-36	activeTask 返回当前活动任务	返回 task-002
TC-37	activeTask 在没有活动任务时返回 null	返回 null
TC-38	selectedStats 返回当前选中的统计数据	返回周维度数据
TC-39	latestLocation 返回最新的位置上报	返回第一条记录
TC-40	latestLocation 在没有上报时返回 null	返回 null
TC-41	copyWith 正确更新字段	更新 selectedStatsPeriod
TC-42	copyWith 的 clearActiveTask 参数可清除活动任务	activeTaskId 为 null

5.8 DriverTaskController 单元测试 (7 个)

编号	用例名称	验证点
TC-43	startTask 成功启动任务并返回成功消息	success=true, 含"已发车"
TC-44	重复 startTask 同一任务返回错误	success=false, 含"已有任务"
TC-45	startTask 不存在的任务返回错误	success=false, "任务不存在"
TC-46	completeActiveTask 成功完成任务	success=true, "任务已完成"
TC-47	没有活动任务时 completeActiveTask 返回错	success=false, "没有进行中任务"

	误	
TC-48	selectStatsPeriod 正确切换统计周期	selectedStatsPeriod 更新
TC-49	simulateNextLocationReport 增加位置上报次数	locationUpdates 长度 +1

5.9 模型与枚举测试 (8 个)

编号	用例名称	验证点
TC-50	DriverTaskModel copyWith 可更新任务状态	status 更新为 running
TC-51	DriverTaskStatus pending 状态值	value="pending", label="待发车"
TC-52	DriverTaskStatus running 状态值	value="running", label="行程中"
TC-53	DriverTaskStatus completed 状态值	value="completed", label="已完成"
TC-54	PassengerPickup 正确存储乘客信息	所有字段正确
TC-55	LocationUpdate 正确存储位置信息	所有字段正确
TC-56	DriverStats 正确存储统计数据	所有字段正确
TC-57	DriverActionResult 正确存储操作结果	success 和 message 正确

6. 测试数据

模拟司机信息:

- 司机姓名: 王师傅
- 车辆 ID: bus-001
- 车牌号: 校 A·1028

模拟任务数据:

任务 ID	路线名称	车次号	起止站点	乘客数	里程
task-001	大学城早班线	D-20260501-01	南门学生公寓 → 综合教学楼	3	8.6 km
task-002	科技园通勤线	D-20260501-02	图书馆广场 → 科技园东门	4	12.4 km
task-003	晚间返校线	D-20260501-03	附属医院站 → 北区宿舍	2	15.8 km

模拟统计数据:

维度	完成车次	乘客数	里程	准点率
日	4 趟	38 人	42.6 km	98%

周	24 趟	236 人	286.4 km	95%
月	98 趟	1034 人	1198.5 km	92%

7. 风险与缓解措施

风险	影响	缓解措施
Timer 导致 pumpAndSettle 无限等待	测试超时	使用 pump 替代 pumpAndSettle
SnackBar 动画时序	测试不稳定	使用 pump 逐步推进动画
滚动列表元素不可见	测试失败	使用 ensureVisible / scrollUntilVisible
Riverpod 状态共享	测试间干扰	每个测试使用独立 ProviderContainer

试用水印

司机端测试样例

1. Widget 测试样例

1.1 司机端登录与工作台展示

登录成功后显示工作台

```
testWidgets('司机端登录成功后显示工作台', (tester) async {  
  await _pumpDriverApp(tester);  
  await _loginAsDriver(tester);  
  expect(find.text('司机端工作台'), findsOneWidget);  
  expect(find.text('统计看板'), findsOneWidget);  
  expect(find.text('实时位置共享'), findsOneWidget);  
  expect(find.text('今日派车任务'), findsOneWidget);  
});
```

工作台显示司机姓名

```
testWidgets('司机端工作台显示司机姓名', (tester) async {  
  await _pumpDriverApp(tester);  
  await _loginAsDriver(tester);  
  expect(find.text('王师傅'), findsOneWidget);  
});
```

初始位置状态为未开启

```
testWidgets('司机端工作台初始位置状态为未开启', (tester) async {  
  await _pumpDriverApp(tester);  
  await _loginAsDriver(tester);  
  expect(find.text('未开启'), findsOneWidget);  
  expect(find.text('位置状态'), findsOneWidget);  
});
```

1.2 统计看板

默认显示日维度数据

```
testWidgets('统计看板默认显示日维度数据', (tester) async {  
  await _pumpDriverApp(tester);  
  await _loginAsDriver(tester);  
  expect(find.text('完成车次'), findsOneWidget);  
  expect(find.text('4 趟'), findsOneWidget);  
  expect(find.text('38 人'), findsOneWidget);  
  expect(find.text('42.6 km'), findsOneWidget);  
});
```

切换到周维度

```
testWidgets('统计看板可切换到周维度', (tester) async {  
  await _pumpDriverApp(tester);  
  await _loginAsDriver(tester);  
  await tester.tap(find.text('周'));  
  await tester.pumpAndSettle();  
  expect(find.text('24 趟'), findsOneWidget);  
  expect(find.text('236 人'), findsOneWidget);  
});
```

```
expect(find.text('286.4 km'), findsOneWidget);
});
```

1.3 任务列表展示

显示路线名称和车次号

```
testWidgets('任务卡片显示路线名称和车次号', (tester) async {
  await _pumpDriverApp(tester);
  await _loginAsDriver(tester);
  expect(find.textContaining('大学城早班线 · D-20260501-01'), findsOneWidget);
  expect(find.textContaining('科技园通勤线 · D-20260501-02'), findsOneWidget);
  expect(find.textContaining('晚间返校线 · D-20260501-03'), findsOneWidget);
});
```

可展开查看乘客名单

```
testWidgets('任务卡片可展开查看乘客名单', (tester) async {
  await _pumpDriverApp(tester);
  await _loginAsDriver(tester);
  await tester.scrollUntilVisible(
    find.text('乘客名单与上车点').first, 200,
    scrollable: find.byType(Scrollable).first,
  );
  await tester.tap(find.text('乘客名单与上车点').first);
  await tester.pumpAndSettle();
  expect(find.text('张同学 · 尾号 0001'), findsOneWidget);
  expect(find.text('陈同学 · 尾号 0132'), findsOneWidget);
  expect(find.text('赵同学 · 尾号 0228'), findsOneWidget);
});
```

1.4 开始任务与位置上报

点击开始任务按钮可启动任务

```
testWidgets('点击开始任务按钮可启动任务', (tester) async {
  await _pumpDriverApp(tester);
  await _loginAsDriver(tester);
  await tester.ensureVisible(find.byKey(const Key('start_task_task-001')));
  await tester.tap(find.byKey(const Key('start_task_task-001')));
  await tester.pumpAndSettle();
  expect(find.text('行程中'), findsOneWidget);
  expect(find.text('共享中'), findsOneWidget);
});
```

模拟上报按钮可手动触发位置上报

```
testWidgets('模拟上报按钮可手动触发位置上报', (tester) async {
  await _pumpDriverApp(tester);
  await _loginAsDriver(tester);
  await tester.ensureVisible(find.byKey(const Key('start_task_task-001')));
  await tester.tap(find.byKey(const Key('start_task_task-001')));
  await tester.pumpAndSettle();
  await tester.ensureVisible(find.byKey(const Key('simulate_location_report')));
  await tester.tap(find.byKey(const Key('simulate_location_report')));
  await tester.pumpAndSettle();
  expect(find.text('上报次数: 2'), findsOneWidget);
});
```

1.5 完成任务

到站完成按钮可结束当前任务

```
testWidgets('到站完成按钮可结束当前任务', (tester) async {
  await _pumpDriverApp(tester);
  await _loginAsDriver(tester);
  await tester.ensureVisible(find.byKey(const Key('start_task_task-001')));
  await tester.tap(find.byKey(const Key('start_task_task-001')));
  await tester.pumpAndSettle();
  await tester.ensureVisible(find.byKey(const Key('complete_active_task')));
  await tester.tap(find.byKey(const Key('complete_active_task')));
  await tester.pumpAndSettle();
  expect(find.text('已完成'), findsOneWidget);
  expect(find.text('未开启'), findsOneWidget);
});
```

完成任务后显示提示消息

```
testWidgets('完成任务后显示提示消息', (tester) async {
  await _pumpDriverApp(tester);
  await _loginAsDriver(tester);
  await tester.ensureVisible(find.byKey(const Key('start_task_task-001')));
  await tester.tap(find.byKey(const Key('start_task_task-001')));
  await tester.pumpAndSettle();
  await tester.ensureVisible(find.byKey(const Key('complete_active_task')));
  await tester.tap(find.byKey(const Key('complete_active_task')));
  await tester.pump();
  await tester.pump(const Duration(milliseconds: 100));
  expect(find.textContaining('任务已完成'), findsWidgets);
});
```

1.6 边界情况

不能同时开始两个任务

```
testWidgets('不能同时开始两个任务', (tester) async {
  await _pumpDriverApp(tester);
  await _loginAsDriver(tester);
  await tester.ensureVisible(find.byKey(const Key('start_task_task-001')));
  await tester.tap(find.byKey(const Key('start_task_task-001')));
  await tester.pumpAndSettle();
  final secondTaskButton = find.byKey(const Key('start_task_task-002'));
  expect(secondTaskButton, findsOneWidget);
  final button = tester.widget<FilledButton>(secondTaskButton);
  expect(button.onPressed, isNull);
});
```

2. 单元测试样例

2.1 DriverTaskState 单元测试

pendingTaskCount 返回待发车任务数量

```
test('pendingTaskCount 返回待发车任务数量', () {
  final state = DriverTaskState(
    tasks: mockTasks, stats: mockStats,
```

```

        selectedStatsPeriod: '日', locationUpdates: const [],
    );
    expect(state.pendingTaskCount, 1);
});

```

activeTask 返回当前活动任务

```

test('activeTask 返回当前活动任务', () {
    final state = DriverTaskState(
        tasks: mockTasks, stats: mockStats,
        selectedStatsPeriod: '日', locationUpdates: const [],
        activeTaskId: 'task-002',
    );
    expect(state.activeTask, isNotNull);
    expect(state.activeTask!.id, 'task-002');
});

```

2.2 DriverTaskController 单元测试

startTask 成功启动任务

```

test('startTask 成功启动任务并返回成功消息', () {
    final container = ProviderContainer(
        overrides: [initialAppModeProvider.overrideWithValue(AppMode.driver)],
    );
    addTearDown(container.dispose);
    final controller = container.read(driverTaskProvider.notifier);
    final result = controller.startTask('task-001');
    expect(result.success, isTrue);
    expect(result.message, contains('已发车'));
    final state = container.read(driverTaskProvider);
    expect(state.activeTaskId, 'task-001');
});

```

completeActiveTask 成功完成任务

```

test('completeActiveTask 成功完成任务', () {
    final container = ProviderContainer(
        overrides: [initialAppModeProvider.overrideWithValue(AppMode.driver)],
    );
    addTearDown(container.dispose);
    final controller = container.read(driverTaskProvider.notifier);
    controller.startTask('task-001');
    final result = controller.completeActiveTask();
    expect(result.success, isTrue);
    expect(result.message, '任务已完成，位置共享已停止');
    final state = container.read(driverTaskProvider);
    expect(state.activeTaskId, isNull);
});

```

3. 辅助方法

```

Future<void> _pumpDriverApp(WidgetTester tester) async {
    await _pumpApp(tester, AppMode.driver);
}

```

```

Future<void> _loginAsDriver(WidgetTester tester) async {
    await _loginWithMockCredential(

```

```
tester, mode: AppMode.driver, loginTitle: '司机端登录',
);
}

Future<void> _pumpApp(WidgetTester tester, AppMode mode) async {
  await tester.pumpWidget(
    ProviderScope(
      overrides: [initialAppModeProvider.overrideWithValue(mode)],
      child: const AppRoot(),
    ),
  );
  await tester.pumpAndSettle();
}

Future<void> _loginWithMockCredential(
  WidgetTester tester, {
    required AppMode mode, required String loginTitle,
  }) async {
  expect(find.text('校车管理系统'), findsOneWidget);
  expect(find.text(loginTitle), findsOneWidget);
  await tester.tap(find.byKey(Key('mock_${mode.value}_login')));
  await tester.pump();
  await tester.ensureVisible(find.byKey(const Key('login_submit')));
  await tester.pump();
  await tester.tap(find.byKey(const Key('login_submit')));
  await tester.pump(const Duration(milliseconds: 600));
  await tester.pumpAndSettle();
}
```

司机端测试报告

1. 测试概要

项目	内容
测试对象	校车管理系统 - 司机端模块
测试类型	Widget 测试 + 单元测试
测试框架	Flutter Test (flutter_test)
测试文件	test/driver_test.dart
执行日期	2026-06-12
测试环境	Flutter SDK 3.x, Windows 11

2. 测试结果总览

00:05 +57: All tests passed!

指标	数值
总测试数	57
通过	57
失败	0
通过率	100%
执行时间	~5 秒

3. 分组测试结果

3.1 Widget 测试

测试组	测试数	通过	失败	通过率
司机端登录与工作台展示	7	7	0	100%
统计看板	4	4	0	100%
任务列表展示	8	8	0	100%
开始任务与位置上报	7	7	0	100%
完成任务	3	3	0	100%
边界情况	5	5	0	100%
小计	34	34	0	100%

3.2 单元测试

测试组	测试数	通过	失败	通过率
DriverTaskState 单元测试	9	9	0	100%
DriverTaskController 单元测试	7	7	0	100%
DriverTaskModel 单元测试	1	1	0	100%
DriverTaskStatus 枚举测试	3	3	0	100%
PassengerPickup 模型测试	1	1	0	100%

LocationUpdate 模型测试	1	1	0	100%
DriverStats 模型测试	1	1	0	100%
DriverActionResult 模型测试	1	1	0	100%
小计	24	24	0	100%

4. 详细测试结果

4.1 司机端登录与工作台展示

通过 - 司机端登录成功后显示工作台

通过 - 司机端工作台显示司机姓名

通过 - 司机端工作台显示待发车任务数量

通过 - 司机端工作台显示今日乘客数量

通过 - 司机端工作台初始位置状态为未开启

通过 - 司机端工作台显示任务列表中的路线名称

通过 - 司机端工作台显示暂无位置上报

4.2 统计看板

通过 - 统计看板默认显示日维度数据

通过 - 统计看板可切换到周维度

通过 - 统计看板可切换到月维度

通过 - 统计看板显示准点率进度条

4.3 任务列表展示

通过 - 任务卡片显示路线名称和车次号

通过 - 任务卡片显示起止站点

通过 - 任务卡片显示车牌号

通过 - 任务卡片显示乘客数量

通过 - 任务卡片显示行驶里程

通过 - 任务卡片显示待发车状态标签

通过 - 任务卡片可展开查看乘客名单

通过 - 乘客名单显示上车点和座位号

4.4 开始任务与位置上报

通过 - 点击开始任务按钮可启动任务

通过 - 开始任务后自动上报一次位置

通过 - 模拟上报按钮可手动触发位置上报

通过 - 多次模拟上报位置坐标会变化

通过 - 开始任务后显示车辆 ID

通过 - 开始任务后显示速度信息

4.5 完成任务

通过 - 到站完成按钮可结束当前任务

通过 - 完成任务后位置上报停止

通过 - 完成任务后显示提示消息

4.6 边界情况

通过 - 未开始任务时模拟上报按钮不可用

通过 - 未开始任务时到站完成按钮不可用

通过 - 不能同时开始两个任务

通过 - 已完成任务显示为不可操作状态

通过 - 完成任务后可以开始另一个任务

4.7 DriverTaskState 单元测试

通过 - pendingTaskCount 返回待发任务数量

通过 - todayPassengerCount 返回所有乘客总数

通过 - activeTask 返回当前活动任务

通过 - activeTask 在没有活动任务时返回 null

通过 - selectedStats 返回当前选中的统计数据

通过 - latestLocation 返回最新的位置上报

通过 - latestLocation 在没有上报时返回 null

通过 - copyWith 正确更新字段

通过 - copyWith 的 clearActiveTask 参数可清除活动任务

4.8 DriverTaskController 单元测试

通过 - startTask 成功启动任务并返回成功消息

通过 - 重复 startTask 同一任务返回错误

通过 - startTask 不存在的任务返回错误

通过 - completeActiveTask 成功完成任务

通过 - 没有活动任务时 completeActiveTask 返回错误

通过 - selectStatsPeriod 正确切换统计周期

通过 - simulateNextLocationReport 增加位置上报次数

4.9 模型与枚举测试

通过 - DriverTaskModel copyWith 可更新任务状态

通过 - DriverTaskStatus pending 状态值为 pending

通过 - DriverTaskStatus running 状态值为 running

通过 - DriverTaskStatus completed 状态值为 completed

通过 - PassengerPickup 正确存储乘客信息

通过 - LocationUpdate 正确存储位置信息

通过 - DriverStats 正确存储统计数据

通过 - DriverActionResult 正确存储操作结果

5. 测试结论

测试结论：通过

- 所有 57 个测试用例全部通过，通过率 100%
- 司机端核心功能（登录、工作台、统计看板、任务管理、位置上报）均验证通过
- 边界情况（重复开始任务、未开始任务操作、已完成任务状态）均正确处理

- 单元测试覆盖了 State、Controller、Model、Enum 等所有关键组件

6. 后续建议

- 添加集成测试：覆盖完整的端到端业务流程
- 启用覆盖率收集：使用 flutter test --coverage 生成覆盖率报告
- 添加性能测试：验证大量任务和位置上报数据下的 UI 性能
- 国际化测试：验证多语言环境下的文本显示
- 无障碍测试：验证屏幕阅读器等辅助功能的兼容性

试用水印